ParkingHub

Código completo explicado
index.html, style.css e script.js

Equipe

Paulo Henrique Alves de Almeida

Matheus Avanzo dos Santos

Murilo Roselini

Petterson Augusto Papa de Souza

Raphael Souza Araujo Barbosa

Cada trecho de código aparece exatamente como está no projeto, com o número das linhas, seguido da explicação do que ele faz.

Antes do código: como tudo se conecta

O ParkingHub é um site feito só com três arquivos. Cada um tem um papel:

Arquivo	Papel	Analogia
`index.html`	Estrutura: quais elementos existem na página.	O esqueleto e as paredes de uma casa.
`style.css`	Aparência: cores, tamanhos, posições.	A pintura e a decoração.
`script.js`	Comportamento: regras, cálculos, validações.	A parte elétrica: faz as coisas funcionarem.

Como se ligam: o HTML dá um `id` a cada elemento importante (ex.: `id="nome"`). O JavaScript encontra o elemento por esse id (`$("nome")`), lê ou altera seu conteúdo e liga e desliga classes e atributos. O CSS reage a essas classes e atributos (ex.: `.spot.sel` fica laranja).

Como está organizado este PDF

Partes 1, 2 e 3: o código dos três arquivos, em trechos, com a explicação logo depois de cada um. **Parte 4:** um dicionário com **cada item** do projeto (todo `id`, função, variável, comando e regra de CSS), para consulta rápida.

Ordem recomendada de explicação

Primeiro o HTML (o que existe), depois o CSS (como parece) e por último o JavaScript (como funciona). No JavaScript, siga a ordem do próprio arquivo: ele está organizado como o usuário usa o site.

Glossário rápido

Termo	Significado
Variável (`const` / `let`)	Um nome que guarda um valor. `const` não pode ser reatribuída; `let` pode.
Função	Um bloco de código com nome, que pode ser chamado várias vezes.
Objeto	Um conjunto de propriedades: `{ nome: "Ana", placa: "ABC1D23" }`.
Array	Uma lista de itens: `[1, 2, 3]`.
Evento	Algo que acontece na página (clique, digitação, envio). `onclick`, `oninput`, `onsubmit` dizem o que fazer quando acontece.
DOM	A página vista pelo JavaScript como uma árvore de elementos que pode ser lida e alterada.
Condição (`if`)	Executa um trecho só se algo for verdadeiro.
Laço (`for`)	Repete um trecho várias vezes, por exemplo para passar por todas as vagas.
localStorage	Pequeno armazenamento do navegador: guarda texto e sobrevive ao recarregar a página.
JSON	Formato de texto para representar objetos e listas (`JSON.stringify` converte para texto; `JSON.parse` volta para objetos).
Milissegundos	Unidade de tempo que o JavaScript usa para datas: 1 hora = 3.600.000 ms.

Parte 1 - index.html

Cabeçalho do documento (head) (linhas 1 a 13)

```
 1  <!doctype html>
 2  <!-- ParkingHub - estrutura da página (versão simples).
 3       O HTML diz O QUE existe na página. Cada elemento importante tem um "id", que o JavaScript usa para
achá-lo.
 4       O atributo "class" serve para o CSS dar o visual. O atributo "hidden" esconde o elemento até o
JavaScript mostrá-lo. -->
 5  <html lang="pt-BR">
 6  <head>
 7    <meta charset="UTF-8">                                              <!-- permite acentos e
caracteres especiais -->
 8    <meta name="viewport" content="width=device-width, initial-scale=1.0">  <!-- ajusta a página à largura
do celular -->
 9    <title>ParkingHub · Reserva de vagas</title>                        <!-- texto da aba do navegador
-->
10    <link rel="icon" href="image/logo-provisoria.jpeg">                <!-- ícone da aba -->
11    <link rel="stylesheet" href="style.css">                              <!-- liga o arquivo de
aparência -->
12    <script src="script.js" defer></script>                               <!-- liga a lógica; "defer"
espera a página carregar antes de rodar -->
13  </head>
```

- `<!doctype html>` avisa ao navegador que é HTML moderno (HTML5).
- `lang="pt-BR"` informa o idioma da página (ajuda leitores de tela e corretor ortográfico).
- `<meta charset="UTF-8">` permite acentos e caracteres especiais.
- `<meta name="viewport">` faz a página se ajustar à largura do celular.
- `<title>` é o texto da aba do navegador.
- `<link rel="icon">` define o ícone da aba. `<link rel="stylesheet">` liga o arquivo `style.css`.
- `<script src="script.js" defer>` liga o JavaScript. O `defer` faz o navegador só executá-lo **depois** de ler a página inteira, para que os elementos já existam quando o JS for procurá-los.

Biblioteca de ícones e cabeçalho da página (linhas 14 a 36)

```
14  <body>
15    <!-- Biblioteca de ícones (invisível): cada <symbol> é um desenho que o resto da página reutiliza com
<use href="#icone-...">.
16         Os ícones usam a cor do texto onde forem colocados. -->
17    <svg width="0" height="0" style="position:absolute" aria-hidden="true">
18      <symbol id="icone-lua" viewBox="0 0 24 24"><path d="M20.8 13.1A9 9 0 0 1 10.9 3.2 9 9 0 1 0 20.8
13.1Z"/></symbol>
19      <symbol id="icone-sol" viewBox="0 0 24 24"><circle cx="12" cy="12" r="4"/><path d="M12 2v2m0 16v2M2
12h2m16 0h2M5 5l1.5 1.5m11 11L19 19M5 19l1.5-1.5m11-11L19 5"/></symbol>
20      <symbol id="icone-moto" viewBox="0 0 24 24"><circle cx="5" cy="17" r="3.5"/><circle cx="19" cy="17"
r="3.5"/><path d="M19 17 15 6h-3m-9 4h5l3 4h5M5 17l4-5m1 2 1 3h4m-7-7 4-2h4"/><path d="M8 10h4l3 3H10Z"
fill="currentColor" stroke="none"/></symbol>
21      <symbol id="icone-acessivel" viewBox="0 0 24 24"><circle cx="11" cy="4" r="1.5"/><path d="M11
7v6h6l3 7m-9-10h6M8 11a6 6 0 1 0 8 8"/></symbol>
22    </svg>
23
24    <!-- CABEÇALHO: logo, nome do sistema e botão que alterna o tema claro/escuro -->
25    <header class="topo">
26      <div class="miolo">
27        <a class="marca" href="">
```

```
28          <img class="logo" src="image/logo-provisoria.jpeg" alt="Logo do ParkingHub">
29          <span>ParkingHub<small>Reserva de vagas</small></span>
30        </a>
31        <!-- Botão de tema: tem os dois ícones e o CSS mostra só um (sol no tema escuro, lua no claro) -->
32        <button class="icone" id="tema" type="button" aria-label="Alternar tema">
33          <svg class="icone-svg icone-sol"><use href="#icone-sol"/></svg><svg class="icone-svg icone-lua">
<use href="#icone-lua"/></svg>
34        </button>
35      </div>
36    </header>
```

- O `<svg>` invisível é a **biblioteca de ícones**: cada `<symbol>` (lua, sol, moto e acessível) é um desenho com um id. Em qualquer lugar, `<use href="#icone-moto">` reaproveita o desenho. Assim não precisamos de emojis nem de arquivos de imagem.
- `<header class="topo">` é o cabeçalho. Dentro dele ficam a logo (`<img>`), o nome "ParkingHub" e o botão `#tema` , que o JS usa para alternar o tema. O botão tem os dois ícones (sol e lua) e o CSS mostra só um.
- `class` serve para o CSS estilizar; `id` é um nome único para o JavaScript achar o elemento.

Título e barra de etapas (linhas 38 a 43)

```
38    <main class="miolo pagina">
39      <h1>Reserve sua vaga</h1>
40      <p class="sub">Escolha sua vaga, informe seus dados e siga para o pagamento.</p>
41
42      <!-- Barra de progresso: o JavaScript marca qual etapa está ativa -->
43      <ol class="etapas" id="etapas"><li>1. Vaga e período</li><li>2. Seus dados</li><li>3. Pagamento</li>
</ol>
```

- `<main>` é o conteúdo principal da página.
- `<ol id="etapas">` é uma lista numerada com as 3 etapas. O JS troca a classe de cada `<li>` para destacar a etapa atual (`atual`) e as já feitas (`feito`).

Etapa 1: o formulário (linhas 45 a 46)

```
45      <!-- ===== ETAPA 1: período, planta e resumo ===== -->
46      <form class="reserva" id="tela-vaga" novalidate>
```

- `<form id="tela-vaga">` agrupa toda a etapa 1. O JS captura o envio (clique em "Continuar") deste formulário.
- Ele tem **duas colunas**: a planta, à esquerda, e uma `<div class="lateral">` com o período e o resumo, à direita. O CSS coloca uma ao lado da outra.

Etapa 1: planta do estacionamento (linhas 48 a 70)

```
48        <!-- Planta: as fileiras estão VAZIAS; o JavaScript cria as 34 vagas dentro delas -->
49        <section class="planta-box">
50          <div class="titulo-planta">
51            <div><h2>Escolha uma vaga</h2><p>Planta do estacionamento</p></div>
52            <span id="disponiveis"></span> <!-- "34 de 34 disponíveis" -->
53          </div>
54          <div class="legenda">
55            <span><i class="livre"></i>Disponível</span>
56            <span><i class="selecionada"></i>Selecionada</span>
57            <span><i class="reservada"></i>Reservada</span>
```

```
 58           <span><svg class="icone-svg" aria-hidden="true"><use href="#icone-acessivel"/></svg>
Acessível</span>
 59         </div>
 60         <div class="rolagem">
 61           <div class="planta">
 62             <div class="muro"></div>
 63             <div class="fileira topo-f" id="fileira-topo"></div>
 64             <div class="fileira meio-f" id="fileira-meio"></div>
 65             <div class="fileira base-f" id="fileira-base"></div>
 66             <div class="entrada"><span>←<small>Saída</small></span><span>→<small>Entrada</small></span>
</div>
 67           </div>
 68         </div>
 69         <p class="legenda-fim">V1–V26: carros · M1–M6: motos · D1–D2: acessíveis</p>
 70       </section>
```

- O título mostra "N de 34 disponíveis" em `#disponiveis` (preenchido pelo JS).
- A legenda usa tags `<i>` vazias, coloridas pelo CSS, como amostras de cor.
- `.rolagem` permite rolar a planta para o lado em telas pequenas.
- Dentro de `.planta` estão o muro desenhado e as **três fileiras vazias** (`#fileira-topo`, `#fileira-meio`, `#fileira-base`). Os botões das 34 vagas **não estão escritos aqui**: o JavaScript cria cada um dentro dessas divs.
- A div `.entrada` desenha as setas de entrada e saída.

Etapa 1: período da reserva (linhas 72 a 108)

```
 72       <!-- Coluna da direita: período e resumo -->
 73       <div class="lateral">
 74
 75       <!-- Período: modalidade, data e horários -->
 76       <section class="periodo">
 77         <h2>Período da reserva</h2>
 78         <fieldset class="modo">
 79           <legend>Modalidade</legend>
 80           <!-- Os dois botões de rádio têm o mesmo "name", então só um pode ficar marcado -->
 81           <label><input type="radio" name="modalidade" value="horario" checked> Por horário</label>
 82           <label><input type="radio" name="modalidade" value="diaria"> Diária de 24 h</label>
 83         </fieldset>
 84         <!-- Data e horários em listas de escolha: assim ficam sempre no formato do Brasil (dia/mês/ano
e 24 horas).
 85              O JavaScript preenche as opções de cada lista. -->
 86         <div class="campo">Data de entrada
 87           <span class="trio">
 88             <select id="dia" aria-label="Dia"></select>
 89             <select id="mes" aria-label="Mês"></select>
 90             <select id="ano" aria-label="Ano"></select>
 91           </span>
 92         </div>
 93         <div class="duas">
 94           <div class="campo">Entrada
 95             <span class="trio">
 96               <select id="entrada-hora" aria-label="Hora de entrada"></select>
 97               <select id="entrada-min" aria-label="Minuto de entrada"></select>
 98             </span>
 99           </div>
100           <div class="campo" id="campo-saida">Saída <!-- some na diária -->
101             <span class="trio">
102               <select id="saida-hora" aria-label="Hora de saída"></select>
103               <select id="saida-min" aria-label="Minuto de saída"></select>
104             </span>
```

```
105           </div>
106         </div>
107         <p class="erro" id="erro-periodo" hidden>Confira a data e os horários: a saída precisa ser
depois da entrada.</p> <!-- aparece quando o período é inválido -->
108       </section>
```

- Dois `<input type="radio">` com o mesmo `name="modalidade"` formam um grupo em que só um pode ser marcado: "horario" ou "diaria". O `value` é o que o JS lê.
- A data e os horários usam **listas de escolha** (`<select>`): dia, mês e ano, e hora e minuto. Os campos de data e hora do navegador (`type="date"` e `type="time"`) aparecem no idioma do navegador (mês/dia/ano e AM/PM), e a página não consegue mudar isso. Com listas, o formato é sempre o do Brasil, com 24 horas. As opções são criadas pelo JavaScript.
- `#campo-saida` é o campo de saída; o JS o esconde quando a modalidade é diária.
- `<p id="erro-periodo" hidden>` é o aviso de período inválido. O atributo `hidden` o mantém oculto até ser necessário.

Etapa 1: resumo e botão continuar (linhas 110 a 126)

```
110       <!-- Resumo: vaga, datas, duração e total (preenchidos pelo JavaScript) -->
111       <section class="resumo-box">
112         <h2>Confira sua reserva</h2>
113         <div class="vaga-escolhida"><span>Vaga selecionada</span><strong id="r-vaga"></strong><small
id="r-tipo"></small></div>
114         <dl>
115           <div><dt>Entrada</dt><dd id="r-entrada"></dd></div>
116           <div><dt>Saída</dt><dd id="r-saida"></dd></div>
117           <div><dt>Duração</dt><dd id="r-duracao"></dd></div>
118           <div class="total"><dt>Total da reserva</dt><dd id="r-total"></dd></div>
119         </dl>
120         <button class="principal" id="continuar" disabled>Selecione uma vaga</button> <!-- liberado só
com período e vaga válidos -->
121         <p class="aviso" id="aviso" role="status" hidden></p>                          <!-- mensagens de
sucesso ou de vaga ocupada -->
122         <p class="nota" id="tarifas"></p>
123       </section>
124
125       </div> <!-- fim da coluna da direita -->
126     </form>
```

- Mostra a vaga selecionada, entrada, saída, duração e total. Cada valor está num `<dd>` com id (`r-vaga` , `r-total` ...) que o JS preenche.
- `<dl>` / `<dt>` / `<dd>` é a lista de definições: nome do dado e seu valor.
- O botão `#continuar` começa `disabled` (desabilitado) e só é liberado quando há período válido e vaga escolhida.
- `#aviso` mostra mensagens (sucesso da reserva, vaga que ficou ocupada) e `#tarifas` mostra os preços.

Etapas 2 e 3: tela de confirmação e resumo (linhas 128 a 144)

```
128     <!-- ===== ETAPAS 2 e 3: o resumo à esquerda é o mesmo; à direita aparece o formulário de dados OU o
de pagamento ===== -->
129     <section class="confirmar" id="tela-confirmar" hidden>
130       <button class="voltar" id="voltar" type="button">← Voltar</button>
131       <div class="duas-colunas">
132
133         <!-- Resumo lateral da reserva -->
```

```
134          <aside class="resumo-box">
135            <h2>Sua reserva</h2>
136            <div class="vaga-escolhida"><strong id="c-vaga"></strong><small id="c-tipo"></small></div>
137            <dl>
138              <div><dt>Modalidade</dt><dd id="c-mod"></dd></div>
139              <div><dt>Entrada</dt><dd id="c-entrada"></dd></div>
140              <div><dt>Saída</dt><dd id="c-saida"></dd></div>
141              <div id="c-cliente" hidden><dt>Cliente</dt><dd id="c-nome"></dd></div> <!-- só aparece na
etapa 3 -->
142              <div class="total"><dt>Total da reserva</dt><dd id="c-total"></dd></div>
143            </dl>
144          </aside>
```

- `#tela-confirmar` começa `hidden`. Ela é compartilhada pelas etapas 2 e 3: o resumo à esquerda é o mesmo e muda só o formulário à direita.
- `<aside>` é o resumo lateral (vaga, modalidade, entrada, saída, cliente, total). A linha do cliente (`#c-cliente`) só aparece na etapa 3.
- O botão "Voltar" tem `type="button"` para não enviar o formulário.

Etapa 2: formulário de dados (linhas 146 a 155)

```
146          <!-- ETAPA 2: dados do cliente e do veículo. "novalidate": a validação é feita pelo nosso
JavaScript -->
147          <form class="form-box" id="form-dados" novalidate>
148            <h2>Seus dados</h2>
149            <label class="campo">Nome completo <input id="nome" maxlength="120" autocomplete="name">
</label>
150            <label class="campo">Telefone com DDD (só números) <input id="telefone" type="tel"
maxlength="11" placeholder="11999998888"></label>
151            <h2>Dados do veículo</h2>
152            <label class="campo">Placa (7 caracteres) <input id="placa" maxlength="7"
placeholder="ABC1D23" autocomplete="off"></label>
153            <p class="erro" id="erro-dados" hidden></p>
154            <button class="principal">Ir para pagamento</button>
155          </form>
```

- Campos de nome, telefone e placa. `maxlength` limita a quantidade de caracteres, `placeholder` mostra o exemplo cinza dentro do campo e `type="tel"` abre o teclado numérico no celular.
- `#erro-dados` exibe a mensagem de erro quando algo está errado.
- `novalidate` desliga a validação automática do navegador, porque a conferência é feita pelo nosso JavaScript, com mensagens em português.

Etapa 3: formulário de pagamento (linhas 157 a 173)

```
157          <!-- ETAPA 3: forma de pagamento (o JavaScript lê qual opção está marcada) -->
158          <form class="form-box" id="form-pagamento" novalidate hidden>
159            <fieldset class="pagamentos">
160              <legend>Forma de pagamento</legend>
161              <label><input type="radio" name="pagamento" value="cartao"> <b>Cartão de crédito</b>
<small>Pagar no estacionamento</small></label>
162              <label><input type="radio" name="pagamento" value="google-wallet"> <b>Google Wallet</b>
<small>Carteira digital</small></label>
163              <label><input type="radio" name="pagamento" value="pix"> <b>Pix</b> <small>Simular com QR
Code</small></label>
164              <label><input type="radio" name="pagamento" value="dinheiro"> <b>Dinheiro</b> <small>Pagar
no estacionamento</small></label>
165            </fieldset>
```

```
166           <h2 id="pg-titulo">Escolha uma opção acima</h2>
167           <p id="pg-texto">O total é o mesmo em todas as formas de pagamento.</p>
168           <img id="pix" src="image/pix-demonstracao.svg" width="180" height="180" alt="QR Code
ilustrativo" hidden> <!-- só no Pix -->
169           <p class="nota">Ambiente de demonstração: nenhuma cobrança real é realizada.</p>
170           <button class="principal" id="pagar" disabled>Confirmar reserva</button>
171         </form>
172       </div>
173     </section>
```

- Quatro `radio` com `name="pagamento"`: cartão, Google Wallet, Pix e dinheiro. O `value` de cada um é a chave usada no objeto `textosPagamento` do JS.
- `<fieldset>` e `<legend>` agrupam e nomeiam as opções.
- O título e o texto (`#pg-titulo`, `#pg-texto`) mudam conforme a opção. A imagem do QR Code (`#pix`) só aparece no Pix.
- O botão `#pagar` começa desabilitado até escolher uma forma. Este formulário começa `hidden`.

Minhas reservas (linhas 175 a 182)

```
175     <!-- ===== MINHAS RESERVAS: a lista é desenhada pelo JavaScript dentro de #lista ===== -->
176     <section class="minhas" id="minhas">
177       <div class="titulo-minhas">
178         <h2>Minhas reservas <span id="contador">0</span></h2> <!-- quantidade de reservas ativas -->
179       </div>
180       <div id="lista"></div>
181     </section>
182   </main>
```

- Título com o contador de reservas ativas (`#contador`).
- `<div id="lista">` começa vazia. O JS desenha ali uma caixa para cada reserva, com o botão de cancelar.

Rodapé (linhas 184 a 198)

```
184   <!-- RODAPÉ: <details> abre e fecha ao clicar, sem precisar de JavaScript -->
185   <footer class="rodape miolo">
186     <span>Projeto de gerenciamento de estacionamento</span>
187     <details><summary>Equipe do projeto</summary>
188       <ul>
189         <li>Paulo Henrique Alves de Almeida</li>
190         <li>Matheus Avanzo dos Santos</li>
191         <li>Murilo Roselini</li>
192         <li>Petterson Augusto Papa de Souza</li>
193         <li>Raphael Souza Araujo Barbosa</li>
194       </ul>
195     </details>
196   </footer>
197 </body>
198 </html>
```

- `<footer>` mostra o nome do projeto e a equipe dentro de `<details>`, um bloco que abre e fecha ao clicar em `<summary>`, sem precisar de JavaScript.

Parte 2 - style.css

Fontes (linhas 1 a 13)

```
/* ParkingHub - aparência do sistema (versão simples)
   O CSS cuida só do visual. O JavaScript liga e desliga classes e atributos,
   e as regras abaixo reagem a eles. */

/* ---------- Fontes (arquivos da pasta fonts) ---------- */
@font-face {
  font-family: "Source Sans 3";
  src: url("fonts/SourceSans3-Variable.ttf");
}
@font-face {
  font-family: "Barlow";
  src: url("fonts/Barlow-SemiBold.ttf");
}
```

- `@font-face` carrega as fontes da pasta `fonts`: Source Sans 3 (textos) e Barlow (títulos). Depois, o nome dado em `font-family` pode ser usado no resto do CSS.

Cores em variáveis e tema escuro (linhas 15 a 53)

```
/* ---------- Cores ----------
   As cores ficam guardadas em variáveis (--nome) e são usadas com var(--nome).
   Para mudar uma cor do site inteiro, basta mudar aqui. */
:root {
  --fundo: #f5f6f7;           /* fundo da página */
  --superficie: #ffffff;      /* fundo de cartões, campos e botões */
  --suave: #e9ecef;           /* cinza de apoio */
  --texto: #191d21;           /* texto principal */
  --secundario: #505c66;      /* texto de apoio */
  --borda: #cdd3d8;           /* bordas */
  --destaque: #c44100;        /* laranja da marca */
  --destaque-suave: #fff1e7;  /* laranja bem claro */
  --sobre-destaque: #ffffff;  /* texto sobre o laranja */
  --piso: #e9ecef;            /* piso do estacionamento */
  --marcacao: #77838e;        /* linhas da planta */
  --reservada: #d6dadd;       /* vagas reservadas */
  --azul: #24669a;            /* vagas acessíveis */
  --vermelho: #ab3522;        /* mensagens de erro */
  --fonte-texto: "Source Sans 3", Arial, sans-serif;
  --fonte-titulo: "Barlow", Arial, sans-serif;
}

/* Tema escuro: quando o <body> ganha a classe "escuro" (o JavaScript faz isso),
   as mesmas variáveis passam a ter cores escuras. */
body.escuro {
  --fundo: #101214;
  --superficie: #16191c;
  --suave: #242a30;
  --texto: #f0f1f2;
  --secundario: #b0bac3;
  --borda: #343b42;
  --destaque: #ff7b29;
  --destaque-suave: #35251b;
  --sobre-destaque: #101214;
  --piso: #0c0f11;
  --reservada: #323940;
  --azul: #88bce8;
```

```
  --vermelho: #ffb0a0;
}
```

- `:root` é a raiz da página. As variáveis `--nome` guardam as cores, e `var(--nome)` usa a cor. Se quisermos mudar o laranja do site inteiro, mudamos uma linha só.
- **Tema escuro:** a regra `body.escuro` só vale quando o `<body>` tem a classe `escuro`. Ela **repete as mesmas variáveis com cores escuras**. O JavaScript liga e desliga essa classe no botão de tema, e todo o site troca de cor sozinho.
- Os comentários ao lado de cada variável dizem para que ela serve.

Estilos gerais da página (linhas 55 a 104)

```
/* ---------- Estilos gerais ---------- */
* {
  box-sizing: border-box;  /* largura e altura incluem borda e espaçamento interno */
}
body {
  margin: 0;
  background: var(--fundo);
  color: var(--texto);
  font-family: var(--fonte-texto);
  font-size: 16px;
  line-height: 1.5;
}
h1, h2, p, dl {
  margin: 0;
}
h1, h2 {
  font-family: var(--fonte-titulo);
}
h1 {
  font-size: 36px;
}
h2 {
  font-size: 22px;
}
button, input, select {
  font-family: inherit;  /* campos usam a mesma fonte da página */
  font-size: inherit;
}
button {
  cursor: pointer;
}
[hidden] {
  display: none !important;  /* quem tem o atributo "hidden" some de verdade */
}

/* Caixa central que limita a largura do conteúdo */
.miolo {
  max-width: 1440px;
  margin: 0 auto;
  padding: 0 32px;
}
.pagina {
  padding-top: 28px;
  padding-bottom: 32px;
}
.sub {
  color: var(--secundario);
  margin-top: 4px;
```

```
  margin-bottom: 20px;
}
```

- `box-sizing: border-box`: a largura e a altura passam a incluir a borda e o espaçamento interno, o que facilita calcular tamanhos.
- `body` define a cor de fundo, a cor do texto, a fonte e o tamanho padrão. `margin: 0` tira a margem que o navegador coloca por padrão.
- `[hidden] { display: none !important; }` garante que qualquer elemento com o atributo `hidden` realmente suma, mesmo que outra regra tente mostrá-lo. O `!important` dá prioridade máxima a essa regra.
- `.miolo` limita a largura do conteúdo (`max-width`) e o centraliza (`margin: 0 auto`).

Cabeçalho, logo e ícones (linhas 106 a 174)

```
/* ---------- Cabeçalho (sempre escuro) ---------- */
.topo {
  background: #000000;
  color: #ffffff;
  padding: 8px 0;
}
.topo .miolo {
  display: flex;                    /* itens lado a lado */
  justify-content: space-between;   /* um à esquerda, outro à direita */
  align-items: center;              /* alinhados ao meio */
}
.marca {
  display: flex;
  align-items: center;
  gap: 12px;
  color: #ffffff;
  text-decoration: none;
  font-family: var(--fonte-titulo);
  font-size: 21px;
}
.marca small {
  display: block;
  font-family: var(--fonte-texto);
  font-size: 12px;
  opacity: 0.7;
}
.logo {
  height: 64px;
}

/* Botão de tema e ícones */
.icone {
  width: 44px;
  height: 44px;
  border: 0;
  background: none;
  color: #ffffff;
}
.icone-svg {
  width: 19px;
  height: 19px;
  flex-shrink: 0;
  fill: none;                 /* só contorno, sem preenchimento */
  stroke: currentColor;       /* cor do traço = cor do texto ao redor */
  stroke-width: 1.7;
  stroke-linecap: round;
```

```
  stroke-linejoin: round;
}
.icone .icone-svg {
  width: 24px;
  height: 24px;
}
.icone-sol {
  display: none;            /* no tema claro mostra a lua */
}
body.escuro .icone-sol {
  display: inline;          /* no tema escuro mostra o sol */
}
body.escuro .icone-lua {
  display: none;
}
.legenda .icone-svg {
  color: var(--azul);
  vertical-align: -4px;
}
.vaga.acessivel .icone-svg {
  width: 25px;
  height: 25px;
}
```

- O cabeçalho é sempre preto (`background: #000000`) com texto branco. Assim a logo, que tem fundo preto, se encaixa sem nenhum truque.
- `display: flex` coloca os filhos **lado a lado**. `justify-content: space-between` empurra um para a esquerda e outro para a direita; `align-items: center` alinha os dois ao meio.
- `.icone-svg` define o tamanho dos ícones. `fill: none` deixa só o contorno e `stroke: currentColor` faz o traço ter a **mesma cor do texto** ao redor.
- Troca sol/lua: por padrão o sol fica escondido (`display: none`). Quando o body tem a classe `escuro` , o sol aparece e a lua some.

Barra de etapas (linhas 176 a 198)

```
/* ---------- Barra de etapas ---------- */
.etapas {
  display: flex;
  gap: 8px;
  padding: 0;
  margin: 0 0 24px 0;
  list-style: none;
}
.etapas li {
  flex: 1;                          /* as 3 etapas dividem o espaço igualmente */
  padding: 8px 0;
  border-top: 3px solid var(--borda);
  color: var(--secundario);
  font-size: 14px;
}
.etapas .atual {                    /* etapa em andamento */
  border-color: var(--destaque);
  color: var(--texto);
  font-weight: bold;
}
.etapas .feito {                    /* etapas já concluídas */
  border-color: var(--texto);
}
```

- A barra é uma lista (`<ol>`) com `display: flex` ; `list-style: none` tira os números automáticos (os números "1.", "2.", "3." estão escritos no HTML).
- `flex: 1` faz as três etapas dividirem o espaço igualmente. Cada uma tem uma borda de cima (`border-top`).
- `.atual` (etapa em andamento) ganha borda laranja e negrito; `.feito` (já concluídas) ganha borda escura. O JavaScript é quem coloca essas classes.

Campos, botões e mensagens (linhas 200 a 283)

```
200 /* ---------- Campos, botões e mensagens ---------- */
201 .campo {
202   display: flex;
203   flex-direction: column;    /* rótulo em cima, campo embaixo */
204   gap: 4px;
205   margin-top: 14px;
206   font-size: 14px;
207 }
208 input, select {
209   padding: 9px 10px;
210   border: 1px solid var(--secundario);
211   border-radius: 3px;
212   background: var(--superficie);
213   color: var(--texto);
214 }
215 fieldset {
216   border: 0;
217   padding: 0;
218   margin: 14px 0 0 0;
219 }
220 .modo label {
221   margin-right: 16px;
222 }
223 .duas {
224   display: flex;
225   gap: 12px;
226 }
227 .duas .campo {
228   flex: 1;
229 }
230 .trio {                      /* várias listas de escolha lado a lado (dia / mês / ano) */
231   display: flex;
232   gap: 6px;
233 }
234 .trio select {
235   flex: 1;
236   min-width: 0;              /* deixa as listas encolherem para caberem na coluna */
237 }
238 .principal {                                  /* botão laranja grande */
239   width: 100%;
240   margin-top: 16px;
241   padding: 12px;
242   border: 0;
243   border-radius: 3px;
244   background: var(--destaque);
245   color: var(--sobre-destaque);
246   font-weight: bold;
247 }
248 .principal:disabled {                         /* botão desabilitado fica cinza */
249   background: var(--suave);
250   color: var(--secundario);
251   cursor: not-allowed;
```

```
}
.botao {                                        /* botão pequeno, com borda */
  padding: 8px 14px;
  border: 1px solid var(--borda);
  border-radius: 3px;
  background: var(--superficie);
  color: var(--texto);
}
.voltar {
  border: 0;
  background: none;
  color: var(--texto);
  padding: 8px 0;
  margin-bottom: 12px;
}
.erro {
  color: var(--vermelho);
  font-size: 14px;
  margin-top: 10px;
}
.aviso {
  margin-top: 12px;
  padding: 10px;
  border-left: 4px solid var(--destaque);
  background: var(--destaque-suave);
  font-size: 14px;
}
.nota {
  color: var(--secundario);
  font-size: 13px;
  margin-top: 8px;
}
```

- `.campo` usa `flex-direction: column`: o rótulo fica em cima e o campo embaixo. `gap` é o espaço entre eles.
- `.duas` coloca dois campos lado a lado (Entrada e Saída). `.trio` coloca várias listas de escolha lado a lado (dia, mês e ano).
- `.principal` é o botão laranja grande. `.principal:disabled` é o mesmo botão quando está desabilitado: fica cinza.
- `.botao` é o botão pequeno com borda. `.erro` é texto vermelho; `.aviso` é a caixa de mensagem com barra laranja à esquerda.

Layout da etapa 1: duas colunas (linhas 285 a 339)

```
/* ---------- Etapa 1: planta à esquerda, período e resumo à direita ---------- */
.reserva {
  display: flex;
  gap: 32px;
  align-items: flex-start;   /* cada coluna tem a altura do próprio conteúdo */
}
.planta-box {
  flex: 1;                   /* ocupa todo o espaço que sobrar */
  min-width: 0;              /* permite a planta rolar para o lado em vez de esticar a página */
  background: var(--superficie);
  border: 1px solid var(--borda);
  overflow: hidden;
}
.lateral {
  width: 344px;              /* coluna da direita (período e resumo) */
}
.titulo-planta {
```

```
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 22px 24px 12px 24px;
}
.titulo-planta p, .titulo-planta span {
  color: var(--secundario);
  font-size: 14px;
}
.legenda {
  display: flex;
  flex-wrap: wrap;           /* se faltar espaço, quebra para a linha de baixo */
  gap: 18px;
  padding: 0 24px 16px 24px;
  color: var(--secundario);
  font-size: 13px;
}
.legenda i {                 /* quadradinhos coloridos da legenda */
  display: inline-block;
  width: 14px;
  height: 14px;
  margin-right: 6px;
  border: 1px solid var(--marcacao);
}
.legenda .livre {
  background: var(--superficie);
}
.legenda .selecionada {
  background: var(--destaque);
}
.legenda .reservada {
  background: var(--reservada);
}
.legenda-fim {
  padding: 14px 24px;
  color: var(--secundario);
  font-size: 12px;
}
```

- `.reserva` usa `display: flex`: a planta e a coluna da direita ficam **lado a lado** com `gap` de 32 px entre elas.
- `.planta-box` tem `flex: 1`: ocupa **todo o espaço que sobrar**. `.lateral` tem largura fixa de 344 px (período e resumo).
- `min-width: 0` permite que a planta role para o lado dentro da caixa, em vez de esticar a página inteira.
- A legenda usa `flex-wrap: wrap`: se não couber, os itens vão para a linha de baixo. Os quadradinhos coloridos são tags `<i>` com tamanho e cor.

A planta e as vagas (linhas 341 a 451)

```
/* ---------- Planta do estacionamento ----------
   As fileiras são posicionadas com porcentagens dentro da planta. */
.rolagem {
  overflow-x: auto;          /* em telas pequenas, rola para o lado */
  border-top: 1px solid var(--borda);
  border-bottom: 1px solid var(--borda);
}
.planta {
  position: relative;        /* as fileiras são posicionadas em relação a ela */
  min-width: 950px;
  height: 740px;
```

```
  background: var(--piso);
}
.muro {                       /* contorno do estacionamento */
  position: absolute;
  top: 3%;
  left: 3%;
  right: 3%;
  bottom: 3%;
  border: 2px solid var(--marcacao);
}
.fileira {
  position: absolute;
  display: flex;              /* as vagas ficam lado a lado, dividindo o espaço */
}
.topo-f {                     /* fileira de cima */
  left: 19%;
  top: 7%;
  width: 69%;
  height: 18%;
}
.meio-f {                     /* fileira do meio: 16 vagas, 8 por linha */
  left: 16%;
  top: 33%;
  width: 78%;
  height: 36%;
  flex-wrap: wrap;
}
.meio-f .vaga {
  width: 12.5%;               /* 8 vagas por linha (100% / 8) */
  height: 50%;                /* 2 linhas */
}
.base-f {                     /* fileira de baixo */
  left: 19%;
  top: 77%;
  width: 69%;
  height: 18%;
}
.motos {                      /* pilha de 3 vagas de moto, uma em cima da outra */
  flex: 1;
  display: flex;
  flex-direction: column;
}
.topo-f .vaga, .base-f .vaga {
  flex: 1;                    /* cada vaga ocupa uma parte igual da fileira */
}
.motos .vaga {
  flex: 1;
}

/* Cada vaga é um botão */
.vaga {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 4px;
  padding: 4px;
  border: 1px solid var(--marcacao);
  background: var(--superficie);
  color: var(--texto);
  font-weight: bold;
}
.vaga.moto {
  flex-direction: row;        /* código e ícone lado a lado */
}
.vaga.acessivel {
```

```
  color: var(--azul);
}
.vaga:hover {                /* mouse em cima */
  background: var(--destaque-suave);
}
.vaga.selecionada {          /* vaga escolhida: laranja */
  background: var(--destaque);
  color: var(--sobre-destaque);
}
.vaga:disabled {             /* reservada ou sem período válido: cinza */
  background: var(--reservada);
  color: var(--secundario);
  cursor: not-allowed;
}
/* As três regras acima têm o mesmo peso; vale a que vier por último. */

.entrada {                   /* setas de entrada e saída na lateral da planta */
  position: absolute;
  left: 2%;
  top: 35%;
  width: 9%;
  height: 32%;
  border-right: 2px dashed var(--marcacao);
  display: flex;
  flex-direction: column;
  justify-content: space-around;
  text-align: center;
  color: var(--secundario);
  font-size: 32px;
}
.entrada small {
  display: block;
  font-size: 11px;
}
```

- `.planta` tem `position: relative`. Dentro dela, o muro e as fileiras têm `position: absolute`, posicionados com `top`, `left`, `width` e `height` em **porcentagem** da planta.
- Cada fileira usa `display: flex`, então as vagas ficam **lado a lado**. Com `flex: 1` cada vaga ocupa uma parte igual da fileira.
- Fileira do meio: tem 16 vagas em 2 linhas. `flex-wrap: wrap` deixa as vagas quebrarem de linha; cada vaga tem `width: 12.5%` (100% dividido por 8 vagas) e `height: 50%` (2 linhas).
- `.motos` é uma coluna (`flex-direction: column`) com 3 vagas de moto, uma em cima da outra.
- **Estados da vaga:** `.vaga` é a normal (branca); `.vaga:hover` destaca ao passar o mouse; `.vaga.selecionada` é laranja; `.vaga:disabled` é cinza (reservada ou sem período válido). Quando duas regras têm o mesmo peso, vale a **última** escrita, por isso a ordem importa.
- `.entrada` desenha as setas de entrada e saída, com uma linha tracejada (`border-right: dashed`).

Período e resumo da reserva (linhas 453 a 492)

```
/* ---------- Período e resumo da reserva ---------- */
.resumo-box {
  margin-top: 24px;
  padding-top: 24px;
  border-top: 1px solid var(--borda);
}
.vaga-escolhida {            /* caixa com o código da vaga */
  margin: 12px 0;
  padding: 12px;
```

```
  background: var(--superficie);
  border: 1px solid var(--borda);
}
.vaga-escolhida span, .vaga-escolhida small {
  display: block;
  color: var(--secundario);
  font-size: 13px;
}
.vaga-escolhida strong {
  font-family: var(--fonte-titulo);
  font-size: 28px;
}
dl div {                    /* cada linha "nome ..... valor" */
  display: flex;
  justify-content: space-between;
  padding: 8px 0;
  border-bottom: 1px solid var(--borda);
  font-size: 14px;
}
dt {
  color: var(--secundario);
}
dd {
  margin: 0;
  font-weight: bold;
  text-align: right;
}
.total dd {
  font-size: 20px;
  color: var(--destaque);
}
```

- `.vaga-escolhida` é a caixa com o código da vaga selecionada.
- Cada linha "nome ..... valor" do resumo é um `<div>` dentro de `<dl>` com `display: flex` e `justify-content: space-between`: o nome fica à esquerda e o valor à direita.
- `.total dd` deixa o valor total maior e laranja.

Layout das etapas 2 e 3 (linhas 494 a 535)

```
/* ---------- Etapas 2 e 3: resumo à esquerda e formulário à direita ---------- */
.duas-colunas {
  display: flex;
  gap: 32px;
  align-items: flex-start;
}
.duas-colunas aside {
  width: 344px;
}
.duas-colunas .resumo-box {
  margin-top: 0;
  padding-top: 0;
  border-top: 0;
}
.form-box {
  flex: 1;
  max-width: 560px;
  padding: 24px;
  background: var(--superficie);
  border: 1px solid var(--borda);
}
.form-box h2 {
  margin-top: 20px;
```

```
}
.form-box h2:first-child {   /* o primeiro título da caixa não precisa de espaço em cima */
  margin-top: 0;
}
.pagamentos label {
  display: block;
  padding: 12px;
  margin-bottom: 8px;
  border: 1px solid var(--borda);
}
.pagamentos small {
  display: block;
  margin-left: 24px;
  color: var(--secundario);
}
#pix {
  display: block;
  margin-top: 12px;
}
```

- `.duas-colunas` também usa `display: flex`: resumo à esquerda (344 px) e formulário à direita (`flex: 1`, no máximo 560 px).
- `.form-box` é a caixa branca do formulário. `.pagamentos label` faz cada opção de pagamento virar um bloco com borda.

Lista de reservas (linhas 537 a 569)

```
/* ---------- Minhas reservas ---------- */
.minhas {
  margin-top: 40px;
}
.titulo-minhas {
  margin-bottom: 12px;
}
#contador {
  padding: 2px 8px;
  border: 1px solid var(--borda);
  background: var(--superficie);
  font-size: 14px;
}
.recibo {                    /* caixa de cada reserva (criada pelo JavaScript) */
  display: flex;
  align-items: center;
  flex-wrap: wrap;
  gap: 16px;
  padding: 16px;
  margin-bottom: 8px;
  background: var(--superficie);
  border: 1px solid var(--borda);
}
.recibo p {
  flex: 1;
  min-width: 220px;
}
.codigo {                    /* código da vaga em laranja */
  min-width: 56px;
  font-family: var(--fonte-titulo);
  font-size: 28px;
  color: var(--destaque);
}
```

- `.recibo` é a caixa de cada reserva (criada pelo JavaScript): o código da vaga, os detalhes (`flex: 1`, ocupam o espaço do meio) e o botão de cancelar. `flex-wrap: wrap` permite quebrar de linha em telas pequenas.
- `.codigo` deixa o código da vaga grande e laranja.

Rodapé (linhas 571 a 578)

```
571  /* ---------- Rodapé ---------- */
572  .rodape {
573    padding-top: 20px;
574    padding-bottom: 32px;
575    border-top: 1px solid var(--borda);
576    color: var(--secundario);
577    font-size: 14px;
578  }
```

- Uma linha separadora em cima (`border-top`), texto menor e em cor de apoio.

Responsividade: celular (linhas 580 a 595)

```
580  /* ---------- Telas menores (até 1000 px): tudo em uma coluna ---------- */
581  @media (max-width: 1000px) {
582    .miolo {
583      padding: 0 16px;
584    }
585    .reserva, .duas-colunas {
586      flex-direction: column;  /* os blocos passam a ficar um embaixo do outro */
587      align-items: stretch;
588    }
589    .lateral, .duas-colunas aside {
590      width: 100%;
591    }
592    .form-box {
593      max-width: none;
594    }
595  }
```

- `@media (max-width: 1000px)` vale só em telas de até 1000 px de largura (tablets e celulares).
- Nelas, `flex-direction: column` troca o "lado a lado" por "um embaixo do outro" na etapa 1 e nas etapas 2 e 3, e as colunas passam a ocupar toda a largura (`width: 100%`).

Parte 3 - script.js

> O arquivo segue a jornada do usuário: variáveis, planta, período e regras, etapa 1, etapa 2, etapa 3, minhas reservas e tema.

Variáveis e tarifas (linhas 1 a 12)

```
1  // ParkingHub - reserva de vagas de estacionamento (versão simples)
2
3  // ----- 1) Variáveis -----
4  const VALOR_HORA = 8;        // tarifas de exemplo, em reais
5  const VALOR_DIARIA = 35;
6  const MS_POR_HORA = 3600000; // o JavaScript guarda datas em milissegundos: 1 hora = 3.600.000 ms
7  const FUSO_BRASIL = "-03:00"; // horário de Brasília: 3 horas atrás do horário mundial (UTC). O Brasil
não tem mais horário de verão.
8
9  let vagas = [];            // as 34 vagas: cada uma é { codigo, tipo, botao }
10 let reservas = [];         // todas as reservas feitas
11 let vagaEscolhida = null;  // vaga clicada na planta (null = nenhuma)
12 let etapaAtual = 1;        // 1 = vaga e período, 2 = seus dados, 3 = pagamento
```

- `const` cria uma variável que **não pode ser trocada** depois. `let` cria uma que pode mudar.
- `VALOR_HORA` e `VALOR_DIARIA` são as tarifas. Para mudar os preços, basta alterar esses dois números.
- `MS_POR_HORA = 3600000`: o JavaScript guarda datas em **milissegundos**. Uma hora tem 60 x 60 x 1000 = 3.600.000 ms. Isso facilita somar e comparar horários.
- `FUSO_BRASIL = "-03:00"`: o horário de Brasília fica 3 horas atrás do horário mundial (UTC). Usamos isso para que todas as datas e horas do sistema sejam **sempre as do Brasil**, mesmo que o computador de quem abre esteja em outro fuso.
- `vagas` e `reservas` são **listas** (arrays). `vagaEscolhida` guarda a vaga clicada (ou `null`, que significa "nada"). `etapaAtual` guarda em qual das 3 etapas estamos.

Salvar e carregar: localStorage (linhas 14 a 20)

```
14 // Carrega as reservas salvas no navegador. O localStorage guarda só texto, então usamos JSON.parse.
15 if (localStorage.getItem("reservas-parkinghub") != null) {
16   reservas = JSON.parse(localStorage.getItem("reservas-parkinghub"));
17 }
18 function salvarReservas() {
19   localStorage.setItem("reservas-parkinghub", JSON.stringify(reservas)); // lista -> texto
20 }
```

- O `localStorage` é uma "gaveta" do navegador que guarda **só texto** e não some ao recarregar a página.
- Ao abrir a página, se existe algo salvo (`!= null`), `JSON.parse` transforma o texto de volta em lista de reservas.
- `salvarReservas()` faz o caminho contrário: `JSON.stringify` transforma a lista em texto e `setItem` guarda com o nome "reservas-parkinghub".

Funções auxiliares (linhas 22 a 29)

```
22  function pegar(id) { return document.getElementById(id); } // atalho: pegar("nome") acha o elemento com id="nome"
23  function moeda(valor) { return "R$ " + valor.toFixed(2).replace(".", ","); }  // 16 -> "R$ 16,00"
24  function dataHora(ms) {  // escreve a data e a hora no formato do Brasil, sempre no horário de Brasília
25    return new Date(ms).toLocaleString("pt-BR", { dateStyle: "short", timeStyle: "short", timeZone: "America/Sao_Paulo" });
26  }
27  function doisDigitos(numero) { if (numero < 10) return "0" + numero; return "" + numero; } // coloca o zero na frente: 5 vira "05"
28  function ehDiaria() { return document.querySelector('input[name="modalidade"]:checked').value == "diaria"; }
29  function icone(nome) { return '<svg class="icone-svg"><use href="#icone-' + nome + '"/></svg>'; } // desenhos do index.html
```

- `pegar("nome")` é um atalho para `document.getElementById("nome")`: devolve o elemento da página que tem aquele `id`. É o que liga o JavaScript ao HTML.
- `moeda(16)` devolve "R$ 16,00": `toFixed(2)` deixa 2 casas decimais e `replace` troca o ponto pela vírgula.
- `dataHora` recebe milissegundos e devolve uma data legível no formato do Brasil, como "26/10/2026, 08:00". A opção `timeZone: "America/Sao_Paulo"` faz o texto sair sempre no horário de Brasília.
- `doisDigitos(5)` devolve "05": coloca o zero na frente quando o número é menor que 10 (usado em datas e horas).
- `ehDiaria()` olha qual botão de modalidade está marcado e devolve `true` ou `false`.
- `icone(nome)` devolve um ícone desenhado no `index.html` (moto ou cadeirante).

criarVaga(): cria o botão de uma vaga (linhas 31 a 47)

```
31  // ----- 2) Planta: 26 vagas de carro (V), 6 de moto (M) e 2 acessíveis (D) -----
32  function criarVaga(codigo, tipo, fileira, classe) {
33    const botao = document.createElement("button");
34    botao.type = "button";
35    botao.className = "vaga " + classe;
36    botao.innerHTML = codigo;
37    if (tipo == "Moto") botao.innerHTML += icone("moto");
38    if (tipo == "Acessível") botao.innerHTML += icone("acessivel");
39
40    const vaga = { codigo: codigo, tipo: tipo, botao: botao };
41    botao.onclick = function () {  // ao clicar: escolhe a vaga (ou desmarca, se já era a escolhida)
42      vagaEscolhida = (vagaEscolhida == vaga) ? null : vaga;
43      atualizar();
44    };
45    vagas.push(vaga);
46    fileira.append(botao);
47  }
```

- As 34 vagas **não estão escritas no HTML**: o JavaScript as cria. `document.createElement("button")` cria um botão novo, `className` define o estilo (CSS) e `innerHTML` o conteúdo: o código da vaga e, se for moto ou acessível, o ícone.
- O objeto `vaga = { codigo, tipo, botao }` junta as informações da vaga. Ele é guardado na lista `vagas` com `push`, e o botão é colocado na fileira com `fileira.append`.
- `botao.onclick = function () {...}` define o que acontece ao clicar: se a vaga já era a escolhida, desmarca; senão, escolhe. O `(condição) ? a : b` é um "se curto". Depois chama `atualizar()` para redesenhar a tela.

Repetindo vagas com laços (linhas 49 a 57)

```
49 function criarMotos(fileira, primeira) {  // pilha de 3 vagas de moto
50   const pilha = document.createElement("div");
51   pilha.className = "motos";
52   fileira.append(pilha);
53   for (let n = primeira; n < primeira + 3; n++) criarVaga("M" + n, "Moto", pilha, "moto");
54 }
55 function criarCarros(fileira, de, ate) {
56   for (let n = de; n <= ate; n++) criarVaga("V" + n, "Carro", fileira, "");
57 }
```

- `criarMotos` cria uma pilha de 3 vagas de moto. `for (let n = primeira; n < primeira + 3; n++)` é um **laço** (repetição) com três partes: onde começa, até quando continua e o que soma a cada volta.
- `"M" + n` junta texto e número: com n = 1 vira "M1".
- `criarCarros` repete de um número até outro para criar V1, V2, V3...

Montando a planta (linhas 59 a 66)

```
59 criarMotos(pegar("fileira-topo"), 1);       // fileira de cima: M1 a M3 e V1 a V6
60 criarCarros(pegar("fileira-topo"), 1, 6);
61 criarVaga("D1", "Acessível", pegar("fileira-meio"), "acessivel");  // fileira do meio: D1, V7 a V13, D2,
V14 a V20
62 criarCarros(pegar("fileira-meio"), 7, 13);
63 criarVaga("D2", "Acessível", pegar("fileira-meio"), "acessivel");
64 criarCarros(pegar("fileira-meio"), 14, 20);
65 criarMotos(pegar("fileira-base"), 4);       // fileira de baixo: M4 a M6 e V21 a V26
66 criarCarros(pegar("fileira-base"), 21, 26);
```

- Cada linha coloca vagas em uma das três fileiras do HTML, na ordem do desenho.
- Cima: M1 a M3 e V1 a V6. Meio: D1, V7 a V13, D2, V14 a V20. Baixo: M4 a M6 e V21 a V26.
- Total: 26 de carro + 6 de moto + 2 acessíveis = **34 vagas**.

calcularPeriodo(): início, fim e valor (linhas 68 a 87)

```
68 // ----- 3) Período, valor e conflitos -----
69
70 // Lê a data e os horários das listas de escolha e devolve { inicio, fim, valor }. Devolve null se
estiver inválido.
71 function calcularPeriodo() {
72   // O JavaScript "corrige" datas que não existem (31/02 viraria 03/03). Por isso conferimos se o dia
existe naquele mês.
73   const existe = new Date(Date.UTC(Number(pegar("ano").value), Number(pegar("mes").value) - 1,
Number(pegar("dia").value)));
74   if (existe.getUTCDate() != Number(pegar("dia").value)) return null;
75
76   const data = pegar("ano").value + "-" + pegar("mes").value + "-" + pegar("dia").value;   // ex.:
"2026-10-26"
77   const horaEntrada = pegar("entrada-hora").value + ":" + pegar("entrada-min").value;       // ex.:
"08:00"
78   const horaSaida = pegar("saida-hora").value + ":" + pegar("saida-min").value;
79   // Juntamos data + hora + fuso (ex.: "2026-10-26T08:00:00-03:00") para o horário ser sempre o de
Brasília.
80   const inicio = new Date(data + "T" + horaEntrada + ":00" + FUSO_BRASIL).getTime();
81   const fim = ehDiaria() ? inicio + 24 * MS_POR_HORA : new Date(data + "T" + horaSaida + ":00" +
FUSO_BRASIL).getTime();
82   if (isNaN(inicio) || isNaN(fim) || fim <= inicio) return null;  // isNaN = "não é número" (data
```

```
inválida)
 83
 84    let valor = VALOR_DIARIA;
 85    if (!ehDiaria()) valor = Math.ceil((fim - inicio) / MS_POR_HORA) * VALOR_HORA;  // ceil arredonda p/
cima: 2h30 = 3 horas
 86    return { inicio: inicio, fim: fim, valor: valor };
 87  }
```

- Lê a data e os horários das listas de escolha. O JavaScript "corrige" datas impossíveis (31/02 viraria 03/03), por isso o código confere se o dia existe naquele mês (`Date.UTC` e `getUTCDate()`) e, se não existir, devolve `null`.
- Depois junta tudo em um texto como "2026-10-26T08:00:00-03:00" (data, hora e fuso de Brasília) e `new Date(...)` entende como **08:00 em Brasília**. O `.getTime()` converte para milissegundos.
- Se a modalidade é diária, o fim é o início + 24 horas. Senão, o fim é a data com o horário de saída.
- `isNaN(inicio)` pergunta "não é um número?". Acontece quando a data está vazia. Se alguma data é inválida ou o fim não é depois do início, devolve `null` (período inválido).
- **Valor:** a diária é fixa. Por horário, `Math.ceil` arredonda as horas **para cima** (2h30 vira 3 horas) e multiplica pela tarifa: 3 x 8 = R$ 24.
- Devolve um objeto `{ inicio, fim, valor }`.

temConflito(): a regra mais importante (linhas 89 a 98)

```
 89  // Já existe reserva ativa com essa vaga (campo = "vaga") ou placa (campo = "placa") nesse período?
 90  // Dois períodos se sobrepõem quando cada um começa antes de o outro terminar.
 91  function temConflito(campo, valor, periodo) {
 92    for (const reserva of reservas) {
 93      if (reserva.status == "ativa" && reserva[campo] == valor) {
 94        if (periodo.inicio < reserva.fim && periodo.fim > reserva.inicio) return true;
 95      }
 96    }
 97    return false;
 98  }
```

- Parâmetros: `campo` ("vaga" ou "placa"), `valor` (qual vaga ou placa) e `periodo` (o período desejado).
- `for (const reserva of reservas)` passa por cada reserva da lista.
- Para cada uma, o primeiro `if` vê se está **ativa** e se é da mesma vaga/placa (`reserva[campo]` lê a propriedade cujo nome está em `campo`).
- O segundo `if` vê se os horários **se sobrepõem**: `periodo.inicio < reserva.fim && periodo.fim > reserva.inicio`, ou seja, cada um começa antes de o outro terminar. Se sim, devolve `true` (tem conflito). Se terminar a lista sem achar, devolve `false`.

Reserva existente	Nova	Conflito?
08h às 10h	09h às 11h	Sim (09 < 10 e 11 > 08)
08h às 10h	10h às 12h	Não (10 não é menor que 10)
08h às 10h	06h às 07h	Não (07 não é maior que 08)

A mesma função serve para a vaga (bloqueia na planta) e para a placa (etapa 2). Reservas **canceladas** são ignoradas, por isso a vaga volta a ficar livre.

atualizar(): redesenha a etapa 1 (linhas 100 a 140)

```
100  // ----- 4) Etapa 1: vaga e período -----
101
102  // Redesenha a etapa 1. Roda toda vez que algo muda (data, horário, vaga clicada).
103  function atualizar() {
104    const periodo = calcularPeriodo();
105    pegar("campo-saida").hidden = ehDiaria();        // na diária não há horário de saída
106    pegar("erro-periodo").hidden = periodo != null;  // aviso só se o período for inválido
107
108    // Se a vaga escolhida ficou ocupada no novo período, desmarca.
109    if (vagaEscolhida != null && periodo != null && temConflito("vaga", vagaEscolhida.codigo, periodo)) {
110      vagaEscolhida = null;
111      pegar("aviso").textContent = "A vaga escolhida está ocupada nesse período. Escolha outra.";
112      pegar("aviso").hidden = false;
113    }
114
115    // Passa por todas as vagas: bloqueia as ocupadas, destaca a escolhida e conta as livres.
116    let livres = 0;
117    for (const vaga of vagas) {
118      const ocupada = periodo != null && temConflito("vaga", vaga.codigo, periodo);
119      vaga.botao.disabled = periodo == null || ocupada;
120      if (vaga == vagaEscolhida) {
121        vaga.botao.classList.add("selecionada");     // pinta de laranja
122      } else {
123        vaga.botao.classList.remove("selecionada");
124      }
125      if (periodo != null && !ocupada) livres++;
126    }
127
128    // Resumo ("—" quando o período é inválido)
129    pegar("disponiveis").textContent = periodo ? livres + " de " + vagas.length + " disponíveis" : "Defina
o período";
130    pegar("r-entrada").textContent = periodo ? dataHora(periodo.inicio) : "—";
131    pegar("r-saida").textContent = periodo ? dataHora(periodo.fim) : "—";
132    pegar("r-duracao").textContent = periodo ? Math.round((periodo.fim - periodo.inicio) / MS_POR_HORA) +
" hora(s)" : "—";
133    pegar("r-total").textContent = periodo ? moeda(periodo.valor) : "—";
134    pegar("r-vaga").textContent = vagaEscolhida ? vagaEscolhida.codigo : "Escolha na planta";
135    pegar("r-tipo").textContent = vagaEscolhida ? vagaEscolhida.tipo : "Carro, moto ou vaga acessível";
136
137    // O botão só é liberado com período válido e vaga escolhida.
138    pegar("continuar").disabled = periodo == null || vagaEscolhida == null;
139    pegar("continuar").textContent = vagaEscolhida ? "Continuar com a vaga " + vagaEscolhida.codigo :
"Selecione uma vaga";
140  }
```

- Roda **toda vez que algo muda** (data, horário ou vaga clicada). Recalcula o período e atualiza a tela.
- `pegar("campo-saida").hidden = ehDiaria()` esconde o campo de saída na diária. `hidden` é um atributo que esconde o elemento.
- Se a vaga escolhida ficou ocupada no novo período, mostra um aviso e desfaz a escolha.
- O laço `for (const vaga of vagas)` passa pelas 34 vagas: calcula se está `ocupada`, bloqueia o botão (`disabled`) se estiver ocupada ou o período for inválido, e, com `if` e `else`, coloca ou tira a classe `selecionada` (laranja) com `classList.add` e `classList.remove`. Também conta as livres.
- O resumo usa `periodo ? A : B`: se existe período, mostra A; senão, mostra B (um traço). A duração é a diferença entre fim e início, dividida por `MS_POR_HORA`.
- O botão "Continuar" só é liberado com período válido **e** vaga escolhida, e o texto dele muda conforme a situação.

mostrarEtapa(): troca de etapa (linhas 142 a 156)

```
142 // Mostra a etapa 1, 2 ou 3 e esconde as outras (o atributo "hidden" esconde o elemento).
143 function mostrarEtapa(numero) {
144   etapaAtual = numero;
145   pegar("tela-vaga").hidden = numero != 1;
146   pegar("tela-confirmar").hidden = numero == 1;
147   pegar("form-dados").hidden = numero != 2;
148   pegar("form-pagamento").hidden = numero != 3;
149   pegar("minhas").hidden = numero != 1;
150   pegar("c-cliente").hidden = numero != 3;
151   const itens = pegar("etapas").children;  // barra de progresso: "atual" na etapa em andamento, "feito"
nas anteriores
152   for (let i = 0; i < itens.length; i++) {
153     itens[i].className = (i + 1 == numero) ? "atual" : (i + 1 < numero) ? "feito" : "";
154   }
155   window.scrollTo(0, 0);
156 }
```

- Recebe o número da etapa (1, 2 ou 3) e mostra/esconde as telas com `hidden`. Ex.: `pegar("tela-vaga").hidden = numero != 1` esconde a etapa 1 sempre que o número não for 1.
- O laço `for` percorre os 3 itens da barra de progresso e dá a classe `atual` (etapa em andamento), `feito` (anteriores) ou nenhuma. É um "se" dentro de outro "se" escrito com `? :`.
- `window.scrollTo(0, 0)` volta ao topo da página.

Eventos da etapa 1 (linhas 158 a 176)

```
158 pegar("tela-vaga").oninput = function () {  // qualquer mudança nos campos redesenha a tela
159   pegar("aviso").hidden = true;
160   atualizar();
161 };
162
163 pegar("tela-vaga").onsubmit = function (evento) {  // botão "Continuar"
164   evento.preventDefault();  // impede o formulário de recarregar a página
165   const periodo = calcularPeriodo();
166   if (periodo == null || vagaEscolhida == null) return;
167   pegar("c-vaga").textContent = vagaEscolhida.codigo;
168   pegar("c-tipo").textContent = vagaEscolhida.tipo;
169   pegar("c-mod").textContent = ehDiaria() ? "Diária de 24 h" : "Por horário";
170   pegar("c-entrada").textContent = dataHora(periodo.inicio);
171   pegar("c-saida").textContent = dataHora(periodo.fim);
172   pegar("c-total").textContent = moeda(periodo.valor);
173   mostrarEtapa(2);
174 };
175
176 pegar("voltar").onclick = function () { mostrarEtapa(etapaAtual - 1); };
```

- `oninput`: roda quando o usuário muda qualquer campo do formulário. Esconde o aviso e chama `atualizar()`.
- `onsubmit` do formulário é o botão "Continuar". `evento.preventDefault()` **impede o comportamento padrão** do formulário (recarregar a página).
- Se o período ou a vaga não estão certos, `return` encerra a função. Se estão, copia os dados para o resumo da etapa 2 e chama `mostrarEtapa(2)`.
- O botão "Voltar" chama `mostrarEtapa(etapaAtual - 1)`.

mostrarErro() (linhas 178 a 183)

```
178 // ----- 5) Etapa 2: seus dados -----
179
180 function mostrarErro(mensagem) {
181   pegar("erro-dados").textContent = mensagem;
182   pegar("erro-dados").hidden = false;
183 }
```

- Escreve a mensagem de erro no parágrafo da etapa 2 e o torna visível.

Validação dos dados (etapa 2) (linhas 185 a 201)

```
185 pegar("form-dados").onsubmit = function (evento) {  // botão "Ir para pagamento"
186   evento.preventDefault();
187   const periodo = calcularPeriodo();
188   const nome = pegar("nome").value.trim();                  // trim() tira os espaços das pontas
189   const telefone = pegar("telefone").value.trim();
190   const placa = pegar("placa").value.trim().toUpperCase();  // toUpperCase() deixa em maiúsculas
191
192   // Confere cada campo. Se algo estiver errado, mostra o erro e para (return).
193   if (!nome.includes(" ")) return mostrarErro("Digite o nome e o sobrenome.");
194   if (isNaN(telefone) || telefone.length < 10 || telefone.length > 11) return mostrarErro("Telefone: só
números, com DDD (10 ou 11 dígitos).");
195   if (placa.length != 7) return mostrarErro("A placa precisa ter 7 caracteres, como ABC1D23.");
196   if (temConflito("placa", placa, periodo)) return mostrarErro("Essa placa já tem uma reserva nesse
período.");
197
198   pegar("erro-dados").hidden = true;
199   pegar("c-nome").textContent = nome + " · " + placa;
200   mostrarEtapa(3);
201 };
```

- Lê os campos. `trim()` remove espaços do começo e do fim; `toUpperCase()` deixa a placa em maiúsculas.
- **Nome:** `nome.includes(" ")` confere se tem um espaço, ou seja, nome e sobrenome. O `!` significa "não".
- **Telefone:** `isNaN(telefone)` é verdadeiro se tiver letras ou símbolos. Depois confere o tamanho com `length`: 10 dígitos (fixo) ou 11 (celular), com DDD.
- **Placa:** precisa ter 7 caracteres (cabe o padrão antigo ABC1234 e o Mercosul ABC1D23).
- **Placa em uso:** `temConflito("placa", placa, periodo)` reaproveita a mesma regra de sobreposição.
- Cada `return mostrarErro(...)` mostra o erro e **para** a função. Se todos os testes passaram, esconde o erro e vai para a etapa 3.

Formas de pagamento (linhas 203 a 215)

```
203 // ----- 6) Etapa 3: pagamento -----
204
205 const textosPagamento = {  // a chave é o "value" de cada opção no HTML
206   cartao: "Cartão de crédito: você paga ao chegar ao estacionamento.",
207   "google-wallet": "Google Wallet: pagamento apenas simulado.",
208   pix: "Pix: veja o QR Code de demonstração (pagamento apenas simulado).",
209   dinheiro: "Dinheiro: você paga ao chegar ao estacionamento."
210 };
211
212 function pagamentoEscolhido() {  // devolve a opção marcada (ex.: "pix") ou null
213   const marcado = document.querySelector('input[name="pagamento"]:checked');
```

```
214    return marcado ? marcado.value : null;
215  }
```

- `textosPagamento` é um objeto: cada chave (cartao, pix...) é igual ao `value` da opção no HTML e guarda o texto que será mostrado. `"google-wallet"` precisa de aspas por ter hífen.
- `pagamentoEscolhido()` procura a opção marcada. Se achar, devolve o `value`; se não, devolve `null`.

Ao escolher o pagamento (linhas 217 a 223)

```
217  pegar("form-pagamento").onchange = function () {  // ao escolher uma forma: mostra o texto, o QR Code (só no Pix) e libera o botão
218    const forma = pagamentoEscolhido();
219    pegar("pg-titulo").textContent = "Forma escolhida";
220    pegar("pg-texto").textContent = textosPagamento[forma];
221    pegar("pix").hidden = forma != "pix";
222    pegar("pagar").disabled = false;
223  };
```

- `onchange` roda quando o usuário escolhe uma opção: mostra o texto da forma escolhida, exibe o QR Code **só se for Pix** e libera o botão "Confirmar".
- Cartão e dinheiro são pagos no estacionamento; Google Wallet e Pix são apenas simulados. Nenhuma cobrança real é feita.

Confirmar: cria a reserva (linhas 225 a 259)

```
225  pegar("form-pagamento").onsubmit = function (evento) {  // botão "Confirmar reserva": cria a reserva
226    evento.preventDefault();
227    const periodo = calcularPeriodo();
228    if (periodo == null || temConflito("vaga", vagaEscolhida.codigo, periodo)) {  // confere de novo se a vaga continua livre
229      mostrarEtapa(1);
230      atualizar();
231      return;
232    }
233
234    const reserva = {  // a reserva é um objeto com todos os dados
235      vaga: vagaEscolhida.codigo,
236      nome: pegar("nome").value.trim(),
237      placa: pegar("placa").value.trim().toUpperCase(),
238      pagamento: pagamentoEscolhido(),
239      inicio: periodo.inicio,
240      fim: periodo.fim,
241      valor: periodo.valor,
242      status: "ativa"  // depois pode virar "cancelada"
243    };
244    reservas.push(reserva);  // push coloca a reserva no final da lista
245    salvarReservas();
246
247    pegar("aviso").textContent = "Reserva da vaga " + reserva.vaga + " confirmada para " + reserva.nome + ". Total: " + moeda(reserva.valor) + ".";
248    pegar("aviso").hidden = false;
249    vagaEscolhida = null;  // limpa tudo para uma nova reserva
250    pegar("form-dados").reset();
251    pegar("form-pagamento").reset();
252    pegar("pix").hidden = true;
253    pegar("pagar").disabled = true;
254    pegar("pg-titulo").textContent = "Escolha uma opção acima";
255    pegar("pg-texto").textContent = "O total é o mesmo em todas as formas de pagamento.";
256    mostrarEtapa(1);
```

```
257    atualizar();
258    listarReservas();
259  };
```

- Antes de criar, **confere de novo** se a vaga continua livre. Se não estiver, volta à etapa 1.
- A reserva é um **objeto** com vaga, nome, placa, forma de pagamento, início, fim, valor e `status: "ativa"`.
- `reservas.push(reserva)` coloca a reserva no final da lista e `salvarReservas()` grava no navegador.
- Depois mostra a mensagem de sucesso, limpa a vaga escolhida e os formulários (`reset()`), volta à etapa 1 e redesenha a planta e a lista.

listarReservas(): desenha a lista (linhas 261 a 293)

```
261  // ----- 7) Minhas reservas -----
262
263  function listarReservas() {  // desenha a lista (a mais nova primeiro)
264    const lista = pegar("lista");
265    lista.innerHTML = "";
266    let ativas = 0;
267    if (reservas.length == 0) lista.textContent = "Nenhuma reserva por enquanto. Selecione uma vaga para
começar.";
268
269    for (let i = reservas.length - 1; i >= 0; i--) {
270      const reserva = reservas[i];
271      if (reserva.status == "ativa") ativas++;
272
273      const caixa = document.createElement("div");  // cada reserva vira uma caixa
274      caixa.className = "recibo";
275      const codigo = document.createElement("strong");
276      codigo.className = "codigo";
277      codigo.textContent = reserva.vaga;
278      const detalhes = document.createElement("p");
279      detalhes.textContent = reserva.nome + " · " + reserva.placa + " · " + dataHora(reserva.inicio) + "
até " +
280        dataHora(reserva.fim) + " · " + moeda(reserva.valor) + " · " + reserva.pagamento + " · " +
reserva.status;
281      caixa.append(codigo, detalhes);
282
283      if (reserva.status == "ativa") {  // só reservas ativas têm o botão de cancelar
284        const botao = document.createElement("button");
285        botao.className = "botao";
286        botao.textContent = "Cancelar reserva";
287        botao.onclick = function () { cancelarReserva(reserva); };
288        caixa.append(botao);
289      }
290      lista.append(caixa);
291    }
292    pegar("contador").textContent = ativas;
293  }
```

- Limpa a lista (`innerHTML = ""`) e, se não há reservas, mostra uma mensagem.
- O laço `for (let i = reservas.length - 1; i >= 0; i--)` anda **de trás para frente**, assim a reserva mais nova aparece primeiro.
- Conta as reservas ativas para o número ao lado do título.
- Cada reserva vira uma caixa criada com `createElement` : o código da vaga, um parágrafo com os detalhes e, **só se estiver ativa**, o botão "Cancelar reserva". Os textos são colocados com `textContent`, que é seguro: se alguém digitar código no nome, ele aparece como texto e não é executado.

cancelarReserva() (linhas 295 a 304)

```
295  function cancelarReserva(reserva) {  // a reserva deixa de estar "ativa", então a vaga volta a ficar
livre
296    if (!confirm("Deseja cancelar a reserva da vaga " + reserva.vaga + "? Clique em OK para confirmar."))
return;
297    reserva.status = "cancelada";
298    salvarReservas();
299    atualizar();
300    listarReservas();
301    // Troca a mensagem antiga ("reserva confirmada") por um aviso de cancelamento.
302    pegar("aviso").textContent = "Reserva da vaga " + reserva.vaga + " cancelada. A vaga está livre de
novo.";
303    pegar("aviso").hidden = false;
304  }
```

- `confirm()` abre uma caixa de "sim ou não". Se o usuário clicar em não, `return` encerra a função.
- Se confirmar, o status vira `"cancelada"`. Como `temConflito` só considera reservas "ativa", a vaga **volta a ficar livre**. Depois salva, redesenha planta e lista e troca a mensagem verde por um aviso de que a reserva foi cancelada.

Tema e início do programa (linhas 306 a 362)

```
306  // ----- 8) Tema e início do programa -----
307
308  // Botão de tema: liga e desliga a classe "escuro" no <body>. O CSS troca as cores por causa dela.
309  pegar("tema").onclick = function () {
310    document.body.classList.toggle("escuro");
311  };
312
313  // Listas de escolha da data e dos horários. Usamos listas (e não os campos de data/hora do navegador)
314  // porque o navegador mostra esses campos no idioma dele (mês/dia/ano e AM/PM). Assim fica sempre no
padrão do Brasil.
315  const NOMES_MESES = ["janeiro", "fevereiro", "março", "abril", "maio", "junho", "julho", "agosto",
"setembro", "outubro", "novembro", "dezembro"];
316
317  function adicionarOpcao(idLista, valor, texto) {  // coloca uma opção (valor e texto) dentro de uma
lista de escolha
318    const opcao = document.createElement("option");
319    opcao.value = valor;
320    opcao.textContent = texto;
321    pegar(idLista).append(opcao);
322  }
323
324  for (let n = 1; n <= 31; n++) {
325    adicionarOpcao("dia", doisDigitos(n), doisDigitos(n));
326  }
327  for (let n = 1; n <= 12; n++) {
328    adicionarOpcao("mes", doisDigitos(n), NOMES_MESES[n - 1]);
329  }
330  for (let hora = 0; hora <= 23; hora++) {      // horas de 00 a 23 (formato de 24 horas)
331    adicionarOpcao("entrada-hora", doisDigitos(hora), doisDigitos(hora));
332    adicionarOpcao("saida-hora", doisDigitos(hora), doisDigitos(hora));
333  }
334  for (let minuto = 0; minuto < 60; minuto += 15) {  // minutos de 15 em 15: 00, 15, 30, 45
335    adicionarOpcao("entrada-min", doisDigitos(minuto), doisDigitos(minuto));
336    adicionarOpcao("saida-min", doisDigitos(minuto), doisDigitos(minuto));
337  }
338
339  // O formulário já começa com a data e a hora de agora no Brasil, assim o período é válido e a planta
não fica bloqueada.
```

```
// Para achar a hora de Brasília, tiramos 3 horas do relógio mundial (UTC) e lemos o resultado com os métodos "getUTC...".
  const agora = new Date(Date.now() - 3 * MS_POR_HORA);
  const horaAtual = agora.getUTCHours();
  const anoAtual = agora.getUTCFullYear();
  adicionarOpcao("ano", anoAtual, anoAtual);          // dois anos para escolher: o atual e o próximo
  adicionarOpcao("ano", anoAtual + 1, anoAtual + 1);
  pegar("dia").value = doisDigitos(agora.getUTCDate());
  pegar("mes").value = doisDigitos(agora.getUTCMonth() + 1);  // os meses começam em 0, por isso o +1
  pegar("ano").value = anoAtual;
  pegar("entrada-hora").value = doisDigitos(horaAtual);       // entrada na hora cheia atual
  pegar("entrada-min").value = "00";
  if (horaAtual < 22) {
    pegar("saida-hora").value = doisDigitos(horaAtual + 2);   // saída 2 horas depois
    pegar("saida-min").value = "00";
  } else {
    pegar("saida-hora").value = "23";                         // perto da meia-noite: a saída fica no fim do mesmo dia
    pegar("saida-min").value = "45";
  }
  pegar("tarifas").textContent = "Valores de exemplo: " + moeda(VALOR_HORA) + " por hora ou " + moeda(VALOR_DIARIA) + " pela diária. Todos os horários seguem o horário de Brasília.";

  mostrarEtapa(1);   // começa na etapa 1
  atualizar();       // desenha a planta e o resumo
  listarReservas();  // mostra as reservas já salvas
```

- O tema escuro é uma **classe no body**. `document.body.classList.toggle("escuro")` liga a classe se ela não existir e desliga se existir. O CSS troca as cores (e o ícone de sol ou lua) por causa dela. Ao abrir, o site começa no tema claro.
- Primeiro o código **cria as opções das listas** com laços `for`: dias de 1 a 31, meses (com o nome escrito), horas de 00 a 23 e minutos de 15 em 15. A função `adicionarOpcao()` cria cada `<option>` e a coloca na lista.
- Depois o formulário começa com a **data e a hora de agora no Brasil**. Para achar a hora de Brasília, tiramos 3 horas do relógio mundial (`Date.now() - 3 * MS_POR_HORA`) e lemos o resultado com os métodos `getUTC...`. A entrada começa na hora cheia atual e a saída 2 horas depois (perto da meia-noite, a saída fica em 23:45 do mesmo dia). Assim o período é válido desde o início e a planta não aparece bloqueada.
- As três últimas linhas executam tudo pela primeira vez: `mostrarEtapa(1)`, `atualizar()` e `listarReservas()` (mostra as reservas já salvas).

Parte 4 - Dicionário: cada item do projeto

> Esta parte lista **tudo** que existe nos três arquivos, item por item. Serve de consulta rápida: se o professor apontar para qualquer nome no código, procure aqui.

4.1 index.html: todos os `id` (nomes que o JavaScript usa)

id	O que é
`icone-lua`, `icone-sol`, `icone-moto`, `icone-acessivel`	Os quatro desenhos de ícone (tags `<symbol>`). São reaproveitados em vários lugares com `<use>`.
`tema`	Botão do cabeçalho que alterna entre tema claro e escuro.
`etapas`	Lista numerada "1. Vaga e período, 2. Seus dados, 3. Pagamento". O JS marca a etapa atual.
`tela-vaga`	Formulário da etapa 1 (período, planta e resumo).
`dia`, `mes`, `ano`	Listas de escolha da data de entrada.
`entrada-hora`, `entrada-min`	Listas de escolha da hora e do minuto de entrada.
`saida-hora`, `saida-min`	Listas de escolha da hora e do minuto de saída.
`campo-saida`	O rótulo e campo da saída. É escondido na diária.
`erro-periodo`	Aviso de que a data ou os horários estão inválidos.
`disponiveis`	Texto "33 de 34 disponíveis".
`fileira-topo`, `fileira-meio`, `fileira-base`	As três fileiras da planta. Começam vazias; o JS cria as vagas dentro delas.
`r-vaga`, `r-tipo`	Resumo da etapa 1: código e tipo da vaga escolhida.
`r-entrada`, `r-saida`, `r-duracao`, `r-total`	Resumo da etapa 1: entrada, saída, duração e valor total.
`continuar`	Botão "Continuar". Fica desabilitado até haver período válido e vaga escolhida.
`aviso`	Mensagens: reserva confirmada ou "vaga ocupada nesse período".
`tarifas`	Texto com os preços de exemplo.
`tela-confirmar`	Tela compartilhada pelas etapas 2 e 3 (resumo ao lado de um formulário).
`voltar`	Botão "Voltar" das etapas 2 e 3.
`c-vaga`, `c-tipo`, `c-mod`, `c-entrada`, `c-saida`, `c-total`	Resumo lateral das etapas 2 e 3: vaga, tipo, modalidade, datas e total.
`c-cliente`, `c-nome`	Linha "Cliente" (nome e placa). Só aparece na etapa 3.
`form-dados`	Formulário da etapa 2.
`nome`, `telefone`, `placa`	Campos que o usuário preenche.
`erro-dados`	Mensagem de erro da etapa 2.
`form-pagamento`	Formulário da etapa 3 com as quatro formas de pagamento.
`pg-titulo`, `pg-texto`	Título e descrição da forma de pagamento escolhida.
`pix`	Imagem do QR Code de demonstração. Só aparece no Pix.

`pagar`	Botão "Confirmar reserva". Começa desabilitado.
`minhas`	Seção "Minhas reservas".
`contador`	Número de reservas ativas ao lado do título.
`lista`	Div onde o JS desenha uma caixa para cada reserva.

4.2 index.html: tags e atributos usados

Item	Para que serve
`<header>`, `<main>`, `<section>`, `<aside>`, `<footer>`	Tags que dão significado às áreas da página (cabeçalho, conteúdo, seção, barra lateral, rodapé).
`<form>`	Agrupa campos e botões. O envio (`submit`) é tratado pelo JS.
`<input type="radio">`	Botão de escolha única. Os que têm o mesmo `name` formam um grupo.
`<select>` e `<option>`	Lista de escolha e cada opção dela. Usadas para a data e a hora, que assim ficam sempre no padrão do Brasil.
`<input type="tel">`	Campo de telefone; abre o teclado numérico no celular.
`<label>`	Rótulo de um campo. Clicar nele ativa o campo.
`<fieldset>` e `<legend>`	Agrupam opções e dão um título ao grupo.
`<dl>`, `<dt>`, `<dd>`	Lista de "nome e valor" (ex.: Entrada: 26/10 08:00).
`<details>` e `<summary>`	Bloco que abre e fecha ao clicar (lista da equipe no rodapé).
`<svg>`, `<symbol>`, `<use>`	Ícones desenhados no HTML e reaproveitados.
`hidden`	Atributo que esconde o elemento até o JS mostrá-lo.
`disabled`	Atributo que desativa um botão ou campo.
`maxlength`, `placeholder`, `autocomplete`	Tamanho máximo de caracteres, exemplo cinza dentro do campo e dica de preenchimento automático.
`novalidate`	Desliga a validação automática do navegador (a conferência é feita pelo nosso JS).
`defer` (no `<script>`)	O JS só roda depois de a página carregar, para os elementos já existirem.

4.3 script.js: toda variável, função e evento

Nome	Tipo	O que faz
`VALOR_HORA`	constante	Preço da hora (R$ 8).
`VALOR_DIARIA`	constante	Preço da diária de 24 h (R$ 35).
`MS_POR_HORA`	constante	Quantos milissegundos tem 1 hora (3.600.000).
`NOMES_MESES`	constante (lista)	Nomes dos 12 meses, para as opções da lista de meses.
`FUSO_BRASIL`	constante	"-03:00": o fuso de Brasília, usado para as datas serem sempre do Brasil.
`vagas`	variável (lista)	As 34 vagas. Cada uma é um objeto `{ codigo, tipo, botao }`.
`reservas`	variável (lista)	Todas as reservas feitas. É carregada e salva no `localStorage`.
`vagaEscolhida`	variável	A vaga clicada na planta, ou `null` se nenhuma.
`etapaAtual`	variável	Em qual etapa estamos: 1, 2 ou 3.
`textosPagamento`	constante (objeto)	Texto de cada forma de pagamento (cartao, google-wallet, pix, dinheiro).
`agora`, `horaAtual`, `anoAtual`	constantes	Data e hora de agora no Brasil, usadas para preencher data, entrada e saída ao abrir.
`salvarReservas()`	função	Grava a lista de reservas no navegador (texto JSON).
`pegar(id)`	função	Atalho para achar o elemento da página com aquele id.
`moeda(valor)`	função	Escreve um número como dinheiro: 16 vira "R$ 16,00".
`dataHora(ms)`	função	Escreve milissegundos como data e hora legíveis, sempre no horário de Brasília.
`adicionarOpcao(...)`	função	Cria uma opção (valor e texto) e a coloca dentro de uma lista de escolha.
`doisDigitos(n)`	função	Coloca o zero na frente de números menores que 10 (5 vira "05").
`ehDiaria()`	função	Devolve `true` se a modalidade marcada é a diária.
`icone(nome)`	função	Devolve o HTML de um ícone (moto, acessível...).
`criarVaga(...)`	função	Cria o botão de uma vaga, define o clique e o coloca na fileira.
`criarMotos(...)`	função	Cria uma pilha de 3 vagas de moto.
`criarCarros(...)`	função	Cria vagas de carro de um número até outro.
`calcularPeriodo()`	função	Lê data e horários e devolve `{ inicio, fim, valor }`, ou `null` se inválido.
`temConflito(...)`	função	Diz se já existe reserva ativa na mesma vaga ou placa com horário sobreposto.

`atualizar()`	função	Redesenha a etapa 1: bloqueia vagas ocupadas, destaca a escolhida e atualiza o resumo.
`mostrarEtapa(n)`	função	Mostra a etapa 1, 2 ou 3 e atualiza a barra de progresso.
`mostrarErro(msg)`	função	Mostra a mensagem de erro da etapa 2.
`pagamentoEscolhido()`	função	Devolve a forma de pagamento marcada, ou `null`.
`listarReservas()`	função	Desenha a lista de "Minhas reservas" e atualiza o contador.
`cancelarReserva(r)`	função	Pede confirmação e muda o status da reserva para "cancelada".
`botao.onclick` (em `criarVaga`)	evento	Ao clicar numa vaga: escolhe ou desmarca e chama `atualizar()`.
`tela-vaga.oninput`	evento	Qualquer mudança nos campos da etapa 1 redesenha a tela.
`tela-vaga.onsubmit`	evento	Botão "Continuar": copia os dados para o resumo e vai para a etapa 2.
`voltar.onclick`	evento	Volta uma etapa.
`form-dados.onsubmit`	evento	Botão "Ir para pagamento": valida nome, telefone e placa.
`form-pagamento.onchange`	evento	Ao escolher a forma de pagamento: mostra o texto, o QR Code (Pix) e libera o botão.
`form-pagamento.onsubmit`	evento	Botão "Confirmar reserva": cria a reserva, salva e volta para a etapa 1.
`tema.onclick`	evento	Alterna entre tema claro e escuro.

4.4 script.js: comandos do JavaScript usados

Comando	O que faz
`const` / `let`	Criam variáveis. `const` não pode ser trocada; `let` pode.
`function nome() {...}`	Define uma função: um bloco de código com nome que pode ser chamado várias vezes.
`return`	Devolve um valor e encerra a função.
`if / else`	Executa um trecho só se a condição for verdadeira.
`cond ? a : b`	"Se curto": se a condição vale, usa a; senão, usa b.
`for (...)` e `for (const x of lista)`	Repetem um trecho. O segundo percorre cada item de uma lista.
`== != < > && || !`	Igual, diferente, menor, maior, "e", "ou" e "não".
`null`	Significa "nada". Usamos quando não há vaga escolhida ou o período é inválido.
`document.getElementById`	Acha um elemento da página pelo id.
`document.querySelector`	Acha o primeiro elemento que combina com um seletor (ex.: o radio marcado).
`document.createElement`	Cria um elemento novo (botão, div...).
`elemento.append`	Coloca um elemento dentro de outro.
`textContent` / `innerHTML`	Mudam o conteúdo de um elemento. `textContent` trata tudo como texto puro (mais seguro).
`className` / `classList.add, remove, toggle`	Trocam as classes de CSS de um elemento. `toggle(classe, condição)` liga ou desliga conforme a condição.
`hidden` / `disabled` / `value`	Esconder, desabilitar ou ler o valor digitado.
`onclick`, `oninput`, `onchange`, `onsubmit`	Eventos: dizem o que fazer quando há clique, digitação, mudança ou envio de formulário.
`evento.preventDefault()`	Impede o formulário de recarregar a página.
`new Date(...)` e `.getTime()`	Criam uma data e a convertem em milissegundos.
`toLocaleString`	Escreve a data no formato do Brasil. Com `timeZone: "America/Sao_Paulo"`, no horário de Brasília.
`Date.now()`, `getUTCHours()`, `getUTCDate()`	Instante atual em milissegundos e as partes da data no horário mundial (UTC). Usados com o ajuste de -3 horas para obter a hora de Brasília.
`Number(x)`, `Date.UTC(...)`	Transformam texto em número e criam uma data no horário mundial (usado para conferir se o dia existe).
`Math.ceil` / `Math.round`	Arredondam para cima / para o mais próximo.
`toFixed(2)`	Deixa o número com 2 casas decimais.

`isNaN(x)`	Verdadeiro se x "não é um número".
`lista.push(x)`	Coloca x no final da lista.
`localStorage.getItem / setItem`	Lê e grava texto na "gaveta" do navegador.
`JSON.stringify / JSON.parse`	Transformam lista ou objeto em texto e de volta.
`confirm()`	Caixa de "sim ou não".
`form.reset()`	Limpa os campos de um formulário.
`window.scrollTo(0, 0)`	Volta ao topo da página.
`trim()`, `toUpperCase()`, `includes()`, `replace()`, `length`	Tirar espaços das pontas, deixar em maiúsculas, ver se o texto contém algo, trocar parte do texto e contar caracteres.
`document.body.classList.toggle("escuro")`	Liga ou desliga a classe "escuro" no body (troca o tema).

4.5 style.css: variáveis de cor e fontes

Variável	Para que serve
`--fundo`	Fundo da página.
`--superficie`	Fundo de cartões, campos e botões.
`--suave`	Cinza de apoio (botão desabilitado).
`--texto`	Cor do texto principal.
`--secundario`	Cor de textos de apoio.
`--borda`	Cor das bordas.
`--destaque`	Laranja da marca (botões, etapa atual, vaga escolhida).
`--destaque-suave`	Laranja bem claro (fundos de destaque e mouse em cima da vaga).
`--sobre-destaque`	Cor do texto sobre o laranja.
`--piso`	Piso do estacionamento na planta.
`--marcacao`	Linhas desenhadas na planta.
`--reservada`	Cinza das vagas reservadas.
`--azul`	Azul das vagas acessíveis.
`--vermelho`	Mensagens de erro.
`--fonte-texto`, `--fonte-titulo`	Fonte dos textos (Source Sans 3) e dos títulos (Barlow).

4.6 style.css: cada regra e seletor

Seletor	O que estiliza
`@font-face`	Carrega as duas fontes da pasta `fonts`.
`:root`	A raiz da página; guarda as variáveis de cor do tema claro.
`body.escuro`	Redefine as variáveis para o tema escuro (vale quando o body tem a classe "escuro").
`*`, `body`, `h1, h2, p, dl`, `button, input, select`	Regras gerais: tamanho das caixas, fundo, fonte, margens e fonte dos campos.
`[hidden]`	Garante que elementos com o atributo `hidden` sumam de verdade.
`.miolo`, `.pagina`, `.sub`	Largura máxima centralizada, espaço da página e subtítulo.
`.topo`, `.marca`, `.logo`	Cabeçalho preto, nome do sistema e logo.
`.icone`, `.icone-svg`, `.icone-sol`, `.icone-lua`	Botão de tema e tamanho e cor dos ícones. Só um dos dois (sol ou lua) aparece.
`.etapas`, `.atual`, `.feito`	Barra de progresso das etapas.
`.campo`, `.duas`, `.trio`, `.modo`	Campos de formulário, dois campos lado a lado e várias listas de escolha lado a lado.
`.principal`, `.botao`, `.voltar`	Botão laranja grande, botão pequeno com borda e botão de voltar.
`.erro`, `.aviso`, `.nota`	Texto de erro (vermelho), caixa de aviso e texto de apoio.
`.reserva`, `.planta-box`, `.lateral`	Layout da etapa 1: planta à esquerda e coluna da direita (período e resumo).
`.titulo-planta`, `.legenda`, `.legenda-fim`	Título da planta, legenda de cores e texto abaixo.
`.rolagem`, `.planta`, `.muro`	Área com rolagem lateral, o piso do estacionamento e o contorno.
`.fileira`, `.topo-f`, `.meio-f`, `.base-f`	As três fileiras de vagas, posicionadas em porcentagem.
`.motos`	Pilha de 3 vagas de moto.
`.vaga`, `.vaga.moto`, `.vaga.acessivel`	Botão de vaga e suas variações (moto em linha, acessível em azul).
`.vaga:hover`	Destaque ao passar o mouse.
`.vaga.selecionada`	Vaga escolhida: fundo laranja.
`.vaga:disabled`	Vaga reservada ou sem período válido: cinza.
`.entrada`	Setas de entrada e saída na lateral da planta.
`.resumo-box`, `.vaga-escolhida`, `dl div`, `dt`, `dd`, `.total`	Resumo: caixa com a vaga e as linhas "nome e valor". O total fica em laranja.

`.duas-colunas`, `.form-box`	Layout das etapas 2 e 3 (resumo e formulário) e a caixa do formulário.
`.pagamentos`, `#pix`	Opções de pagamento e a imagem do QR Code.
`.minhas`, `.titulo-minhas`, `#contador`	Seção de reservas, título e o número ao lado dele.
`.recibo`, `.recibo p`, `.codigo`	Caixa de cada reserva (criada pelo JS), seus detalhes e o código da vaga em laranja.
`.rodape`	Rodapé.
`@media (max-width: 1000px)`	Em telas pequenas, os blocos ficam um embaixo do outro.

4.7 style.css: propriedades e conceitos que aparecem

Item	O que faz
`var(--nome)`	Usa o valor de uma variável.
`display: flex`	Coloca os filhos lado a lado (ou um embaixo do outro, com `flex-direction: column`).
`justify-content`, `align-items`	Alinham os filhos no sentido da linha e no sentido contrário. `space-between` empurra para as pontas; `center` centraliza.
`flex: 1`, `flex-wrap: wrap`, `gap`	Ocupar o espaço que sobrar, quebrar de linha se faltar espaço e espaço entre os itens.
`position: relative / absolute`	Posiciona um elemento dentro de outro. A planta usa `absolute` com porcentagens.
`top`, `left`, `right`, `bottom`	Distância do elemento até cada borda da caixa onde ele está posicionado.
`width`, `height`, `max-width`, `min-width`	Tamanhos (largura e altura) e seus limites.
`margin`, `padding`	Espaço de fora e espaço de dentro de uma caixa.
`border`, `border-radius`	Borda e cantos arredondados.
`background`, `color`	Cor de fundo e do texto.
`font-family`, `font-size`, `font-weight`	Fonte, tamanho e peso (negrito).
`:hover`, `:disabled`, `:first-child`	Estados: mouse em cima, desabilitado e primeiro filho.
`overflow-x: auto`	Cria rolagem lateral quando o conteúdo não cabe.
`display: none`	Esconde o elemento.
`stroke: currentColor`	Faz o traço do ícone usar a mesma cor do texto.
`px`, `%`	Unidades: pixels e porcentagem do elemento pai.

Resumo para fechar a apresentação

- **Fluxo:** escolher período, escolher vaga, preencher dados, pagar, ver em "Minhas reservas".
- **Regras principais:** `calcularPeriodo()` calcula o valor; `temConflito()` impede reservas sobrepostas de vaga e de placa; `cancelarReserva()` libera a vaga de novo.
- **Dados:** tudo em um array `reservas` salvo em `localStorage` como JSON. Sem servidor.
- **Interface:** o JS só liga e desliga classes e atributos; o CSS decide o visual (inclusive o tema escuro, por variáveis).
- **Limites:** é um protótipo: pagamento simulado, dados só no navegador de quem usou.